Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy.

Operate as one coordinated team. Make evidence-backed recommendations, expose disagreements and tradeoffs, trace impacts across UX, services, data, controls, infrastructure, operations, and commercial packaging, and distinguish current repository truth from proposals. Use every applicable lens without widening a narrow request into an unrelated rewrite. Mark immaterial lenses `not applicable` with one short reason. Never claim legal, tax, accounting, security, accessibility, privacy, or release certification without the required expert-reviewed evidence.

Workspace: E:\ohada saas\Focused projects\stoquify

Primary roadmap: E:\ohada saas\Focused projects\stoquify\what-next\codebase-rewrite\execution-roadmap.md

Supporting evidence directory: E:\ohada saas\Focused projects\stoquify\what-next\codebase-rewrite\

1. Mission

Using the existing roadmap and repository evidence, establish a comprehensive, practical execution method for completing the entire codebase review and justified rewrite from its current state through final verification and handover.

Translate the roadmap into an operational process that determines:

- What work happens next and why.
- What must be true before it starts.
- Who owns implementation, review, integration, and verification.
- Which activities can proceed independently.
- How results are measured and recorded.
- How failures, blockers, conflicting changes, and interruptions are handled.
- How work resumes without repeating discovery or losing decisions.
- What evidence establishes package, stage, and whole-program completion.

Produce the working execution documents, structured backlog, and reusable instructions. Validate the process through a documented dry run.

This assignment establishes the execution method. Application rewrites and production operations are outside this assignment.

2. Establish current truth

Read:

- Applicable AGENTS.md instructions.

- execution-roadmap.md.
- baseline.md, rewrite-plan.md, progress.md, verification.md, reviewer-coverage.md, and final-report.md.
- Relevant evidence files, command logs, source hashes, and benchmark methods.
- Package manifests, lockfiles, build/test scripts, CI configuration, deployment configuration, and relevant migration/test setup.
- Relevant source, graphify-out/, what-next/, and innovation/ material.

Treat reports as dated evidence. Verify the facts needed to design the method against current files and configuration.

Reconcile the last recorded state:

- Two implementation stages already exist.
- The attributable reduction was 1,745 application lines.
- The focused suites and final typecheck passed for recorded source hashes.
- The isolated production build timed out after 15 minutes.
- Local application probes timed out.
- The inventory gate reported 20 findings outside those edits.
- Real-database performance and authenticated browser validation remain open.

Do not assume these results still apply to changed inputs. Preserve valid prior work and evidence; refresh only what is stale or insufficient.

3. Preserve complete scope and reviewer coverage

Map every roadmap stage, 00–10, into the execution method: 00: Reliable verification. 01: Complete product and workflow inventory. 02: Architecture decisions and measurable targets. 03: Shared foundations. 04: Inventory, POS, and offline workflows. 05: Commercial and payment workflows. 06: Accounting, close, and statutory boundaries. 07: HRIS and payroll. 08: Remaining product areas. 09: Measured optimization and removal of superseded code. 10: Integrated acceptance and handover.

Use all ten core and twelve domain review responsibilities defined by the aqstoqflow-prompt-architect skill and existing reviewer-coverage.md.

Assign responsibilities to affected work packages. Distinguish implementation, technical review, and any required qualified human review. Do not invent approvals or represent different review perspectives as independent sign-offs.

Account for every maintained application area, supported workflow, dynamic entry point, and externally consumed contract. Retained modules require an evidenced review and applicable verification.

4. Define the execution structure

Establish three levels:

- Program: complete scope, priorities, dependencies, targets, and final acceptance.
- Stage: related work packages and an explicit exit gate.
- Work package: one coherent, reviewable unit of behavior or architecture.

For each stage specify:

- Objective and included scope.

- Entry criteria and dependencies.
- Implementation sequence.
- Required reviewer responsibilities.
- Verification and evidence requirements.
- Exit criteria.
- Blocker, recovery, and rollback paths.

Create a dependency graph. Identify the critical path and work that can proceed independently.

Allow stages 00 and 01 to progress independently where appropriate. Do not allow unresolved dependencies to be silently bypassed, and do not let one blocked environment task prevent unrelated discovery.

5. Define a standard work-package contract

Every package must contain:

- Stable ID, title, roadmap stage, and business outcome.
- Scope, files/boundaries owned, and explicit exclusions.
- Required behavior, consumers, and public contracts.
- Dependencies and readiness conditions.
- Implementation, review, and integration responsibilities.
- Risk and affected security/data-integrity invariants.
- Baseline identity and measurable acceptance criteria.
- Planned implementation and verification commands.
- Test data, environment, and resource requirements.
- Migration, consumer transition, and rollback approach.
- Temporary adapters and their removal conditions.
- Evidence links, status, blockers, and exact next action.

Separate disposition from progress:

- Disposition: retain, consolidate, replace, or remove.
- Progress: unreviewed, inventoried, characterized, ready, implementing, verifying, verified, or blocked.

Define the evidence required for every status transition. A retained package is not verified merely because it produced no code changes.

6. Establish the repeatable execution procedure

Specify the procedure for each package:

A. Inspect its current implementation, consumers, dependencies, and authority boundaries. B. Separate required behavior from confirmed defects and unsupported assumptions. C. Capture the baseline and establish acceptance budgets. D. Add focused behavioral protection where existing evidence is inadequate. E. Implement the smallest coherent change across affected boundaries. F. Verify business behavior, negative authorization cases, failure/recovery, integration contracts, and relevant performance. G. Migrate consumers and confirm dynamic entry points. H. Remove superseded implementations when their removal conditions are met. I. Review the final diff, measurements, and evidence. J. Update the register and select the next eligible package.

Specify how an authorized execution run continues through eligible work without requiring routine reconfirmation.

Require user input only when a material decision, missing authority, or unavailable requirement genuinely prevents safe progress. Prepare the concrete options and evidence before requesting that decision.

7. Control concurrency and resource use

Define a practical work-in-progress limit and explain its basis.

For any parallel implementation:

- Require independent scopes and explicit file/contract ownership.
- Define the handoff contract and integration owner.
- Prevent simultaneous incompatible edits to shared contracts.
- Use isolated checkouts when necessary.
- Preserve accepted uncommitted changes when preparing an isolated snapshot; starting from HEAD alone may omit the implemented work.

Serialize resource-heavy builds, typechecks, benchmarks, or database-mutating tests on a shared host unless resource and environment isolation are demonstrated.

Assign distinct ports, build directories, browser contexts, and fixture identities where needed. Track which resources belong to each run.

Parallelism must reduce elapsed work without making results unreliable or creating merge/review bottlenecks. Review roles do not automatically require separate agents or additional architectural layers.

8. Establish verification and evidence rules

Build a verification matrix:

- Package checks for changed behavior.
- Stage checks for shared contracts and domain integration.
- Program checks for cross-domain workflows and release acceptance.

Discover actual repository scripts before prescribing commands. Where applicable, include:

- npm run typecheck
- npm run policy:gates
- npm run workflow:assurance:runtime-check
- npm run prisma:validate
- npm run build:app

Add focused tests, browser/accessibility checks, database integration tests, contract checks, recovery exercises, and benchmarks according to affected risk.

Inspect commands before running them against a database or external service. Identify the target environment and use dedicated non-production fixtures.

For each result record:

- Package and run IDs.
- Source snapshot/hash and relevant dependency/configuration identity.
- Command, environment, dataset, and benchmark conditions.
- Start/end times, exit code, and result classification.

- Log/artifact paths.
- Scope limitations and evidence invalidation conditions.

Use distinct results: passed, failed, timed out, blocked, and not run.

Reuse passing evidence only when relevant inputs and assumptions remain valid. Rerun affected checks after changes; avoid repeating unrelated expensive checks. Preserve original baselines and historical evidence.

Measure correctness/security first, clarity second, performance third, and size fourth. Do not equate deleted lines or passing-test counts with program completion.

9. Define blocker handling and recovery

For each blocker record:

- Symptom and reproducible evidence.
- Affected packages and unaffected work.
- Current diagnosis and uncertainty.
- Responsible role.
- Next diagnostic or remediation action.
- Conditions that allow execution to resume.

Address the existing blockers explicitly:

- Build timeout: distinguish resource contention, configuration, dependency fan-out, and application compilation problems.
- Browser availability: establish a responsive test application and valid dedicated identities.
- Inventory findings: classify and remediate each finding without erasing historical evidence or using blanket exclusions.
- Database verification: establish a reproducible dataset and valid comparison method before claiming endpoint or PostgreSQL performance gains.

Retry only when there is a changed input, a new diagnostic hypothesis, or evidence of a transient failure. Avoid repeatedly issuing the same failing command.

Define task-scoped rollback and cleanup. Preserve unrelated user changes, processes, build outputs, data, and evidence. Stop only resources owned by the run.

For schema changes, specify compatible migration stages, reconciliation, consumer transition, recovery validation, and temporary-layer removal.

10. Make execution resumable and auditable

Create one authoritative work register. Derive summaries from it rather than maintaining conflicting status lists.

At every package boundary and interruption, record:

- Last verified package and source identity.
- Current package and unfinished changes.
- Commands completed and processes still running.
- Evidence that remains valid or has become stale.
- Decisions, assumptions, blockers, and temporary adapters.

- The exact next action and its prerequisites.

Define a startup/resume procedure that checks this state against the current workspace before continuing. Another engineer or agent must be able to resume without reconstructing the entire conversation.

Keep reporting concise: completed work, measured results, blockers, next eligible work, and decisions needed. Clearly separate completed implementation from completed verification.

11. Produce the execution artifacts

Save the method under: E:\ohada saas\Focused projects\stoquify\what-next\codebase-rewrite\

Reuse compatible existing artifacts. Select one canonical register format and avoid duplicate sources of truth.

Required outputs:

- execution-method.md: Operating procedure, stage gates, ownership, scheduling, concurrency, verification, blocker handling, rollback, and startup/resume instructions.
- work-packages.json: Validated package records, dependencies, readiness, status, and evidence links.
- evidence-index.json: Links between packages, source identities, checks, results, and limitations.
- handoff.md: Current state and exact continuation instructions.
- method-validation.md: Dry-run scenarios, discovered weaknesses, corrections, and remaining limits.

Update existing progress.md, verification.md, and reviewer-coverage.md only as needed to remain consistent. Preserve historical results.

Include reusable instructions for:

- Starting the next eligible package.
- Resuming interrupted work.
- Reviewing and closing a package.
- Handling a blocked package.
- Performing final program acceptance.

Populate stages 00–02 with concrete initial packages based on current evidence. Represent later packages at an honest level of detail and specify how discovery will refine them. Do not manufacture a fully analyzed backlog.

12. Validate the method before declaring it ready

Perform a documented dry run using actual initial package IDs and known evidence.

Exercise:

- Successful package completion and next-package selection.
- The build timeout and progression of independent work.
- A failed authority or financial-invariant check.
- A source change that invalidates previously passing evidence.
- Conflicting edits or competing heavy processes.
- An interruption followed by resume.
- A retained module with insufficient verification.
- Final acceptance with one unresolved required gate.

Check that dependency references exist, ordering is consistent, status transitions require evidence, and the next action can be determined without guesswork.

Label simulated scenarios as simulated. Do not convert dry-run outcomes into claims that application checks passed.

13. Success criteria and boundaries

The execution method is ready when:

- Every roadmap stage has an executable procedure and completion gate.
- Every known blocker has a concrete resolution path.
- Initial packages are actionable and dependencies are explicit.
- Responsibilities, resource use, handoffs, and integration are defined.
- Evidence reuse and invalidation rules prevent stale success claims.
- Interruption and recovery do not lose decisions or duplicate completed work.
- Completion requires every supported area to have a verified disposition.
- The dry run demonstrates usable start-to-finish control of the process.
- A new engineer or agent can identify and execute the next permitted action from the saved artifacts.

Do not claim the codebase rewrite is complete because the method is complete. Do not invent whole-platform reduction percentages or calendar commitments. Estimate schedule only from the discovered backlog, dependencies, capacity, measured throughput, and uncertainty.

Preserve supported capabilities, server-owned business truth, tenant isolation, RBAC, entitlements, monetary precision, transaction boundaries, idempotency, audit evidence, redaction, and applicable statutory provenance.

Do not touch unrelated lint warnings, introduce speculative features, replace frameworks without evidence, build a custom orchestration application, deploy to production, or weaken verification gates.

Begin by reconciling the roadmap with current evidence. Deliver the operational method, working registers, validated initial work queue, and concise instructions for executing the first milestone.